Processes, Threads and CPU Scheduling

Operating Systems - Units 1-2 - Process management

What does an operating system do?

An operating system manages hardware resources (CPU, memory, storage, devices), provides abstractions such as processes and files, and offers a safe interface to programs through system calls.

Processes

A process is a program in execution with its own address space, registers and open files. The OS tracks each process in a Process Control Block (PCB).

- States: new, ready, running, waiting (blocked), terminated.
- A context switch saves the state of the running process and loads another - it is pure overhead.
- fork() creates a child process; exec() replaces its program image; wait() lets the parent collect the exit status.

Threads

Threads are lightweight units of execution inside a process. They share code, heap and files but have separate stacks and registers, so creation and switching are cheaper than for processes - but shared data needs synchronization.

CPU scheduling criteria

- CPU utilization and throughput - maximize.
- Turnaround time = completion - arrival. Waiting time = turnaround - burst. Response time = first run - arrival.

Scheduling algorithms

- FCFS: simple, non-preemptive, suffers from the convoy effect.
- SJF / SRTF: minimum average waiting time but needs burst estimates and can starve long jobs.
- Round Robin: each process gets a time quantum; good response time; too small a quantum causes overhead.
- Priority: highest priority first; use aging to prevent starvation.

> *Worked example (FCFS): P1 burst 5, P2 burst 3, P3 burst 8, all arriving at 0. Waiting times are 0, 5 and 8, so the average waiting time is 13/3 = 4.33.*

Synchronization

A race condition occurs when the result depends on thread timing. A critical section must offer mutual

exclusion, progress and bounded waiting. Tools: mutex locks, semaphores (wait/signal) and monitors. Classic problems: producer-consumer, readers-writers, dining philosophers.